main.py

Share Run

```python
class Car:
    make = "Toyota"
    model = "Camry"
    year = 2022

# Create object
my_car = Car()

# Print attributes
print("Make:", my_car.make)
print("Model:", my_car.model)
print("Year:", my_car.year)
```

Output

```
Make: Toyota
Model: Camry
Year: 2022

=== Code Execution Successful ===
```

main.py

Share Run

```python
class Car:
    make = "Toyota"
    model = "Camry"
    year = 2022

    def start_engine(self):
        print(f"The {self.year} {self.make} {self.model}'s engine has started! 🚗")

# Create object
my_car = Car()

# Call method
my_car.start_engine()
```

Output

```
The 2022 Toyota Camry's engine has started! 🚗

=== Code Execution Successful ===
```

```python
class Student:
    def __init__(self, name, age):   # Constructor
        self.name = name
        self.age = age

# Create object
s1 = Student("Alice", 20)

# Print attributes
print("Name:", s1.name)
print("Age:", s1.age)
```

```
Name: Alice
Age: 20

=== Code Execution Successful ===
```

```python
class Animal:
    def __init__(self, name, species):
        self.name = name
        self.species = species

class Dog(Animal):          # Dog inherits from Animal
    def __init__(self, name, species, breed):
        super().__init__(name, species)   # Call parent constructor
        self.breed = breed

# Create Dog object
d = Dog("Buddy", "Canine", "Labrador")

print("Name:", d.name)
print("Species:", d.species)
print("Breed:", d.breed)
```

```
Name: Buddy
Species: Canine
Breed: Labrador

=== Code Execution Successful ===
```

```python
class A:
    def show(self):
        print("Show from A")

class B(A):
    def show(self):
        print("Show from B")

class C(A):
    def show(self):
        print("Show from C")

class D(B, C):   # Inherits from both B and C
    pass

d = D()
d.show()

# Check the Method Resolution Order (MRO)
print("MRO:", [cls.__name__ for cls in D.__mro__])
```

```
Show from B
MRO: ['D', 'B', 'C', 'A', 'object']

=== Code Execution Successful ===
```

```python
class Shape:
    def __init__(self, color):
        self.color = color

class Circle(Shape):
    def __init__(self, color, radius):
        super().__init__(color)    # Initialize parent's
            attribute
        self.radius = radius

# Create object
c = Circle("Red", 7)

print("Color:", c.color)
print("Radius:", c.radius)
```

```
Color: Red
Radius: 7

=== Code Execution Successful ===
```

```python
class Person:
    def __init__(self, name):
        self.name = name

class Employee:
    def __init__(self, employee_id):
        self.employee_id = employee_id

class Manager(Person, Employee):
    def __init__(self, name, employee_id):
        Person.__init__(self, name)          # Explicitly call each parent
        Employee.__init__(self, employee_id)

# Create object
m = Manager("Bob", "EMP101")

print("Name:", m.name)
print("Employee ID:", m.employee_id)
```

```
Name: Bob
Employee ID: EMP101

=== Code Execution Successful ===
```

```python
import math

class Shape:
    def area(self):
        pass   # To be overridden

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return round(math.pi * self.radius ** 2, 2)

class Square(Shape):
    def __init__(self, side):
        self.side = side

    def area(self):
        return self.side ** 2

# List of Shape objects
shapes = [Circle(5), Square(4), Circle(3)]

for shape in shapes:
    print(f"{shape.__class__.__name__} area: {shape.area()}")
```

```
Circle area: 78.54
Square area: 16
Circle area: 28.27

=== Code Execution Successful ===
```

```python
from abc import ABC, abstractmethod

class Vehicle(ABC):          # Abstract Base Class
    @abstractmethod
    def start_engine(self):  # Abstract method — must be
        implemented
        pass

class Car(Vehicle):
    def start_engine(self):
        print("Car engine started — Vroom! ")

class Bike(Vehicle):
    def start_engine(self):
        print("Bike engine started — Brap! ")

# Create objects
c = Car()
b = Bike()

c.start_engine()
b.start_engine()
```

```
Car engine started — Vroom!
Bike engine started — Brap!

=== Code Execution Successful ===
```

```python
class Walker:
    def walk(self):
        print("Walking... ")

class Runner:
    def run(self):
        print("Running... ")

class Athlete(Walker, Runner):   # Inherits from BOTH
    pass

# Create object
a = Athlete()
a.walk()
a.run()
```

```
Walking...
Running...

=== Code Execution Successful ===
```